iND83213A CS01 40leds SDK Release Note

V1.14

1 REVISION HISTORY

Table 1 Revision History

Rev #	Date	SDV Version and Action	By
1.00	10/07/2023	Ver: libdev_iND83213A_CS01_App_B0.01.02_lin_2.0.8_colorlib_2.2.2_40leds (1)Make adc clock to be 4MHz	Winter
1.01	10/11/2023	Ver: libdev_iND83213A_CS01_App_B0.01.03_lin_2.0.8_colorlib_2.2.2_40leds (1) Add ADC Data Filter Function for negative value	Winter
1.02	10/20/2023	Ver: libdev_iND83213A_CS01_App_B0.01.04_lin_2.0.8_colorlib_2.2.2_40leds (1) ADC negative processing before calling adcCallback	Winter
1.03	10/26/2023	Ver: libdev_iND83213A_CS01_App_B0.01.05_lin_2.0.8_colorlib_2.2.2_40leds (1) update lin default params for SNPD	Winter
1.04	12/01/2023	Ver: ibdev_iND83213A_CS01_App_B0.01.06_lin_2.0.8_colorlib_2.2.2_40leds (1) Fix an issue where the temperature reading was abnormal when the blue LED was lit alone	Eric
1.05	12/19/2023	Ver: libdev_iND83213A_CS01_App_B0.01.07_lin_2.0.8_colorlib_2.2.2_40leds (1) Porting linslaveTask_Private.c	Winter
1.06	3/21/2024	Ver: libdev_iND83213A_CS01_App_B0.01.08_lin_2.0.8_colorlib_2.2.2_40leds (1) Update LIN Performance Params for LIN_AA (2) Fix sleep issue @Private_linsavetask mode	Winter
1.07	4/2/2024	Ver: libdev_iND83213A_CS01_App_B0.01.09_lin_2.0.9_colorlib_2.2.2_40leds (1) Update LIN Performance Params for LIN_AA (2) Fix pwm device IRQ API (3) lin tack lib update J2602 Error Report logic @J2602 Mode	Winter
1.08	4/19/2024	Ver: libdev_iND83213A_CS01_App_B0.01.10_lin_2.0.9_colorlib_2.2.2_40leds (1) Resolve the issue of abnormal sleep mode branches	Levi Ye
1.09	7/26/2024	Ver: libdev_iND83213A_CS01_App_B0.01.11_lin_2.1.0_colorlib_2.2.2_40leds (1) Fix the issue of report led error when it is turned off (2) Add an linstack api for Once Force 3D response without 3C require (3) Add special functional NAD (0x7E) handler	Levi Ye

Rev #	Date	SDV Version and Action	By
1.10	7/30/2024	Ver: libdev_iND83213A_CS01_App_B0.01.12_lin_2.1.0_colorlib_2.2.3_40leds (1) ADD HSL&RGB Color Mixing CMDS @ 40LEDS APP	Levi Ye
1.11	9/29/2024	Ver: libdev_iND83213A_CS01_App_B0.01.13_lin_2.1.0_colorlib_2.2.3_40leds (1) Update PUM_EnterDeepSleeepMode(void) API for reduce sleep current	Winter
1.12	6/12/2024	Ver: libdev_iND83213A_CS01_App_B0.01.14_lin_2.1.0_colorlib_2.2.3_40leds (1) Fix the issue of Buck Set Freq Api Inaccurate settings @buckoperation.a	Levi Ye
1.13	1/2/2025	Ver: libdev_iND83213A_CS01_App_B0.01.15_lin_2.1.0_colorlib_2.2.3_40leds (1) Update Temperature Calculation Method (2) Add Handling Negative Voltage	Levi
1.14	2/28/2025	Ver: libdev_iND83213A_CS01_App_B0.01.16_lin_2.1.0_colorlib_2.2.3_40leds (1) Fix the issue of abnormal wake-up. (2) Fix the issue of led false fault report by two-point calibration (3) Fix the issue of system clock division mismatch	Levi

2 TABLE OF CONTENTS

1 REVISION HISTORY 2
2 TABLE OF CONTENTS 4
3 V1.14 6
3.1 ChangeVersion 6
3.2 Bug Fix 6
3.2.1 Fix the issue of abnormal wake-up. 6
3.2.2 Fix the issue of led false fault report by two-point calibration 7
3.2.3 Fix the issue of system clock division mismatch 8
4 V1.13 10
4.1 ChangeVersion 10
4.2 Bug Fix 10
4.2.1 Update Temperature Calculation Method 10
4.2.2 Add Handling Negative Voltage 11
5 V1.12 12
5.1 ChangeVersion 12
5.2 Bug Fix 12
5.2.1 Fix the issue of Buck Set Freq Api Inaccurate settings 12
6 V1.11 13
6.1 Version Change 13
6.2 Feature Update 13
6.2.1 Update PMU_EnterDeepSleepMode(void) API 13
7 V1.10 14
7.1 Version Change 14
7.2 NEW FEATURE 14
7.2.1 ADD HSL&RGB Color Mixing CMDS @ 40LEDS APP 14
8 V1.09 15
8.1 Version Change 15
8.2 Bug Fix 15
8.2.1 Fix the issue of reporting led error when it is turned off 15
8.3 NEW Feature 16
8.3.1 Add an linstack api for Once Force 3D response without 3C require. 16
8.3.2 Add special functional NAD (0x7E) handler. 17
9 V1.08 18
9.1 Version Change 18
9.2 BUG Fix 18
9.2.1 Resolve the issue of abnormal sleep mode branches 18
10 V1.07 19

10.1 ChangeVersion **19**
10.2 LIN stack Update **19**
10.2.1 Update error report logic @J2602 Mode 19
10.3 BUG FIXED **20**
10.3.1 Update LIN device Params for improve LIN_AA performance 20
10.3.2 Fix pwm device driver API error 20

11 V1.06 ..... **21**
11.1 ChangeVersion **21**
11.2 BUG FIXED **21**
11.2.1 Update LIN device Params for improve LIN_AA performance 21
11.2.2 Fix sleep issue @Private_linsavetask mode 22

12 V1.05 ..... **23**
12.1 ChangeVersion **23**
12.2 BUG Fixed **23**
12.2.1 Porting linslaveTask_Private.c 23

13 V1.04 ..... **24**
13.1 ChangeVersion **24**
13.2 BUG Fixed **24**
13.2.1 Set blue intensity threshold value before PN detection 24

14 V1.03 ..... **25**
14.1 ChangeVersion **25**
14.2 Default Status Modify **25**
14.2.1 Default Params Modify For SNPD 25

15 V1.02 ..... **26**
15.1 ChangeVersion **26**
15.2 BUG Fixed **26**
15.2.1 ADC negative processing before calling adcCallback 26

16 V1.01 ..... **27**
16.1 ChangeVersion **27**
16.2 BUG Fixed **27**
16.2.1 None 27
16.3 New Feature **28**
16.3.1 Add ADC Code Filter 28

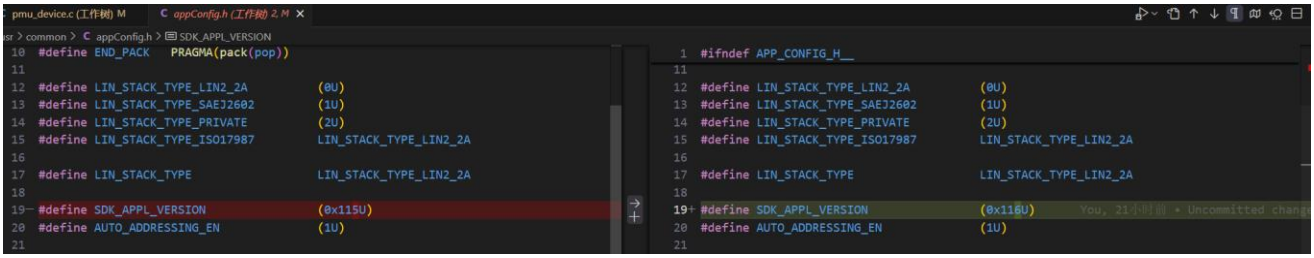
17 V1.00 ..... **29**
17.1 ChangeVersion **29**
17.2 BUG Fixed **29**
17.2.1 Make ADC clock source to be 4MHz 29

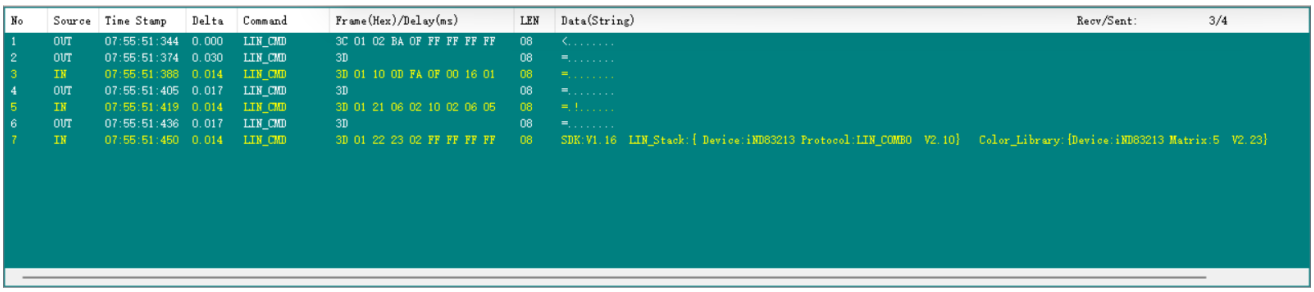
3 V1.14

3.1 CHANGEVERSION

Form Version: libdev_iND83213A_CS01_App_B0.01.15_lin_2.1.0_colorlib_2.2.2_40leds

Update Version: libdev_iND83213A_CS01_App_B0.01.16_lin_2.1.0_colorlib_2.2.3_40leds





3.2 BUG FIX

3.2.1 Fix the issue of abnormal wake-up.

3.2.1.1 issue description:

When using sleep mode and designating LINS as the wake-up source, it is necessary to disable LINM wake-up enable. Before executing LINM_WUP_ENA = 0, be sure to clear LINM_Sleep.

3.2.1.2 solution:

Make sure linm sleep = 0 when entering sleepmode：

```c
void PMU_EnterDeepSleepMode(void)
{
    PMUAPRE5V_SFRS->CFG_ACCESS.CFG_ACCESS_KEY = 0x0BU;
    BUCK_DisableBuck();                    /* disable buck*/
    CRGA_SFRS->MODULERSTREQ     |= MODULE_SOFT_RESET_PWM | MODULE_SOFT_RESET_ADC | M
    LINS_SFRS->CTRL.SLEEP       = 0U;

    PMUAPRE5V_SFRS->LINS.SWON               = 0U;

    /* disbale temperature sensor when sleep */
    SAR_CTRL_SFRS->OT_CTRL_CFG.ADC_SAMP_OT_ENA = 0U;
    SAR_CTRL_SFRS->OT_CTRL_CFG.EN_OVERTEMP = 0U;

    PMUA_SFRS->VBATCTRL.BAT_UV_EN = 0U;
    PMUA_SFRS->VBATCTRL.BAT_OV_EN = 0U;

    SYSCTRLA_SFRS->BIAS.LEDBIASTRIMEN = 0U;
    PMUAPRE5V_SFRS->CTRL.PD1V5_ENA_HIBERNATE = 1U;
    /* clear all of wake up flags */

    /* Enable wakeup detect and IRQ  when LINS is in sleep mode(LINS_REG_CTRL.SLEEP
    WICA_SFRS->CTRL.LINSIRQCLR = 1U;
    WICA_SFRS->CTRL.LINSIRQENA = 1U;
    NVIC_EnableIRQ(WICA_IRQn);
    PMUAPRE5V_SFRS->LINS.LINS_RX_BIAS_BOOST = 0U;
```

```c
void PMU_EnterDeepSleepMode(void)
{
    PMUAPRE5V_SFRS->CFG_ACCESS.CFG_ACCESS_KEY = 0x0BU;
    BUCK_DisableBuck();                    /* disable buck*/
    CRGA_SFRS->MODULERSTREQ     |= MODULE_SOFT_RESET_PWM | MODULE_SOFT_RESET_ADC | MODUL
    LINS_SFRS->CTRL.SLEEP       = 0U;
    LINM_SFRS->CTRL.SLEEP       = 0U;

    PMUAPRE5V_SFRS->LINS.SWON               = 0U;

    /* disbale temperature sensor when sleep */
    SAR_CTRL_SFRS->OT_CTRL_CFG.ADC_SAMP_OT_ENA = 0U;
    SAR_CTRL_SFRS->OT_CTRL_CFG.EN_OVERTEMP = 0U;

    PMUA_SFRS->VBATCTRL.BAT_UV_EN = 0U;
    PMUA_SFRS->VBATCTRL.BAT_OV_EN = 0U;

    SYSCTRLA_SFRS->BIAS.LEDBIASTRIMEN = 0U;
    PMUAPRE5V_SFRS->CTRL.PD1V5_ENA_HIBERNATE = 1U;
    /* clear all of wake up flags */
    /* Disable linm wakeup detect and IRQ (LINM_REG_CTRL.SLEEP = 0U;)*/
    WICA_SFRS->CTRL.LINMIRQCLR = 1U;
    WICA_SFRS->CTRL.LINMIRQENA = 0U;
    /* Enable lins wakeup detect and IRQ  when LINS is in sleep mode(LINS_REG_CTRL.SLEEP
    WICA_SFRS->CTRL.LINSIRQCLR = 1U;
    WICA_SFRS->CTRL.LINSIRQENA = 1U;
    NVIC_EnableIRQ(WICA_IRQn);
    PMUAPRE5V_SFRS->LINS.LINS_RX_BIAS_BOOST = 0U;
```

3.2.2 Fix the issue of led false fault report by two-point calibration

3.2.2.1 issue description:

1) Fix LED false fault report due to LED PN Voltage after two-point calibration
2) When the calculated PN junction voltage is negative, it will exist in the form of a complement code, while the open circuit voltage detection is 4000U. Therefore, when the voltage is calculated to be negative, such as -1, it will be stored as 65535, which will cause false alarms

3.2.2.2 solution:

1) Store the calculated PN voltage as int type, and also store the short-circuit detection voltage as int type. When negative pressure occurs, there will be no error message

```c
int16_t get_volt(AdcDatabuff_t *const param)
{
    int16_t volt = (int16_t)INVALID_VOLT;
    int32_t sVolt;
    uint32_t uVolt;
    uint32_t sum = 0;
    if (param->startCode != INVALID_PARAM){
        for (uint8_t i = 0U; i < AVERAGE_MEASURE_GAIN; i++){
            sum += param->buff[i];
        }
        sum  = sum >> (AVERAGE_MEASURE_POS - 1U);
        sVolt = param->calibP.gain * (int32_t)sum + param->calibP.offset;
        if (sVolt <= 0){
            sVolt = 0;

        }
        uVolt = (uint32_t)sVolt;
        uVolt = uVolt >> MEASURE_GAIN_POS;
        volt = (int16_t)uVolt;

    }
    return volt;
}
```

```c
#if (LED_NUM <= 8U)
int16_t get_volt(AdcDatabuff_t *const param)
{
    int16_t volt = (int16_t)INVALID_VOLT;
    int32_t sVolt;
    uint32_t uVolt;
    uint32_t sum = 0;
    if (param->startCode != INVALID_PARAM){
        for (uint8_t i = 0U; i < AVERAGE_MEASURE_GAIN; i++){
            sum += param->buff[i];
        }
        sum  = sum >> (AVERAGE_MEASURE_POS - 1U);
        sVolt = param->calibP.gain * (int32_t)sum + param->calibP.offset;
        if(sVolt <=0){
            sVolt = -sVolt;
            uVolt = (uint32_t)sVolt;
            uVolt = uVolt >> MEASURE_GAIN_POS;
            volt = (int16_t)uVolt;
            volt = -volt;
        }else{
            uVolt = (uint32_t)sVolt;
            uVolt = uVolt >> MEASURE_GAIN_POS;
            volt = (int16_t)uVolt;
        }
    }
    return volt;
}
```

```c
/*
 *  Get Runtime Led PN voltage in mV
 *  @param [out] pnVoltR/pnVoltG/pnVoltB: 0-4000mV
 *  @return :none
 */
void MES_GetRunTimeLedPNVolt(LedNum_t ledIndex,uint16_t *pnVoltR, uint16_t *pnVoltG,uint16_t *pnVoltB
{
    *pnVoltR = (uint16_t)adcResult.vLedPN[ledIndex][0].target;
    *pnVoltG = (uint16_t)adcResult.vLedPN[ledIndex][1].target;
    *pnVoltB = (uint16_t)adcResult.vLedPN[ledIndex][2].target;
}
```

```c
#if (LED_NUM <= 8U)
/*
 *  Get Runtime Led PN voltage in mV
 *  @param [out] pnVoltR/pnVoltG/pnVoltB: 0-4000mV
 *  @return :none
 */
void MES_GetRunTimeLedPNVolt(LedNum_t ledIndex,int16_t *pnVoltR, int16_t *pnVoltG,int16_t *pnVoltB)
{
    *pnVoltR = adcResult.vLedPN[ledIndex][0].target;
    *pnVoltG = adcResult.vLedPN[ledIndex][1].target;
    *pnVoltB = adcResult.vLedPN[ledIndex][2].target;
}
```

```c
#define OVER_TEMPERATURE_HIGH           (130)   /*celsius degree */
#define OVER_TEMPERATURE_RELEASE        (OVER_TEMPERATURE_HIGH - 10)   /*celsius degree */


#define LED_PN_SHORT_VOLT                       (500U)   /* mV */
#define LED_PN_SHORT_OPEN                       (4000U)  /* mV */

#define INTENSITY_RATIO_TYPICAL                 (1024U)                         /* 100% of ma
#define INTENSITY_RATIO_HIGH_TEMPERATURE        (INTENSITY_RATIO_TYPICAL*4U/5U)         /* 80% of ma
#define INTENSITY_RATIO_BATT_ISSUE              (0U)                            /* 0% of max
```

```c
#define OVER_TEMPERATURE_HIGH           (130)   /*celsius degree */
#define OVER_TEMPERATURE_RELEASE        (OVER_TEMPERATURE_HIGH - 10)   /*celsius degree */


#define LED_PN_SHORT_VOLT                       (500)   /* mV */
#define LED_PN_OPEN_VOLT                        (4000)  /* mV */

#define INTENSITY_RATIO_TYPICAL                 (1024U)                         /* 100% of max i
#define INTENSITY_RATIO_HIGH_TEMPERATURE        (INTENSITY_RATIO_TYPICAL*4U/5U)         /* 80% of max int
#define INTENSITY_RATIO_BATT_ISSUE              (0U)                            /* 0% of max int
```

```c
static void safetyHandle(void)
{
    /*  Led PN Voltage Protection Handling     */
    uint16_t pnVoltR,pnVoltG,pnVoltB;
    uint8_t pnState;
    for (uint8_t ledIndex = 0U; ledIndex < LED_NUM; ledIndex++){
        pnState = 0U;
        MES_GetRunTimeLedPNVolt((LedNum_t)ledIndex,&pnVoltR, &pnVoltG,&pnVoltB);
        if(PWM_LedIsOn(ledIndex) == 0U){
            pnVoltR = INVALID_VOLT;
            pnVoltG = INVALID_VOLT;
            pnVoltB = INVALID_VOLT;
        }
        if ( (pnVoltR != INVALID_VOLT) && (pnVoltG != INVALID_VOLT)  && (pnVoltB != INVALID_VOLT) ){
            if ( (pnVoltR <= LED_PN_SHORT_VOLT) || (pnVoltG <= LED_PN_SHORT_VOLT) || (pnVoltB <= LED_
                pnState = 1U;           /* short  */
            }
            if ( (pnVoltR >= LED_PN_SHORT_OPEN) || (pnVoltG >= LED_PN_SHORT_OPEN) || (pnVoltB >= LED_
                pnState |= (1U << 1);   /* open  */
            }
        }
        ledPNState[ledIndex] = (LedPNState_t)pnState;
    }
```

```c
static void safetyHandle(void)
{
    /*  Led PN Voltage Protection Handling     */
    int16_t pnVoltR,pnVoltG,pnVoltB;
    uint8_t pnState;
    for (uint8_t ledIndex = 0U; ledIndex < LED_NUM; ledIndex++){
        pnState = 0U;
        MES_GetRunTimeLedPNVolt((LedNum_t)ledIndex,&pnVoltR, &pnVoltG,&pnVoltB);
        if(PWM_LedIsOn(ledIndex) == 0U){
            pnVoltR = INVALID_VOLT;
            pnVoltG = INVALID_VOLT;
            pnVoltB = INVALID_VOLT;
        }
        if ( (pnVoltR != INVALID_VOLT) && (pnVoltG != INVALID_VOLT)  && (pnVoltB != INVALID_VOLT) ){
            if ( (pnVoltR <= LED_PN_SHORT_VOLT) || (pnVoltG <= LED_PN_SHORT_VOLT) || (pnVoltB <= LED_PN_S
                pnState = 1U;           /* short  */
            }
            if ( (pnVoltR >= LED_PN_OPEN_VOLT) || (pnVoltG >= LED_PN_OPEN_VOLT) || (pnVoltB >= LED_PN_OPE
                pnState |= (1U << 1);   /* open  */
            }
        }
        ledPNState[ledIndex] = (LedPNState_t)pnState;
    }
```

3.2.3 Fix the issue of system clock division mismatch

3.2.3.1 issue description:

1) The actual frequency division on the datasheet is the register value+1, while the frequency division value set in the code is the actual frequency division value=2 ^ register value

3.2.3.2 solution:

1) Modify the frequency division value macro definition in the code to be consistent with the datasheet

```c
#define CLOCK_DIV_1  (0U)
#define CLOCK_DIV_2  (1U)
#define CLOCK_DIV_4  (2U)
#define CLOCK_DIV_8  (3U)
```

```c
#ifndef CLOCK_DEVICE_H__
#define CLOCK_DIV_1  (0U)
#define CLOCK_DIV_2  (1U)
#define CLOCK_DIV_4  (3U)
#define CLOCK_DIV_8  (7U)
```

clock_device.c (工作树) 2, M

drivers > hal > src > clock_device.c > Clock_SystemMainClockInit(uint8_t)

```c
void Clock_SystemMainClockInit(uint8_t divider)
{
    CRGA_SFRS->SYSCLKCTRL.HFRCENA = 1U;
    while(CRGA_SFRS->SYSCLKCTRL.HFRCSTS == 0U){}
    /* Enable trim access write enable */
    SYSCTRLA_SFRS->TRIM_ACCESS_KEY.TRIM_ACCESS_KEY = 0x0EU;
    SYSCTRLA_SFRS->HF_OSC_TRIM.TRIM_HF_RC = HWCFG_GetRCHFClockCalibValue();
    PMUAPRE5V_SFRS->CLK_CTRL.TRIM_LF_RC = HWCFG_GetRCLFClockCalibValue();
    CRGA_SFRS->LFCLKCTRL.CLKLFSEL = 1;
    /* Disable trim access write enable */
    SYSCTRLA_SFRS->TRIM_ACCESS_KEY.TRIM_ACCESS_KEY = 0U;
    CRGA_SFRS->SYSCLKCTRL.SYSCLKSEL = (uint8_t)CLOCK_RC_16MHz;
    CRGA_SFRS->SYSCLKCTRL.DIVSYSCLK = (uint8_t)divider;
    /*  frequency spread    */
    SYSCTRLA_SFRS->HF_OSC_TRIM.SSCDEEP  = 7U;/*  3  */
    SYSCTRLA_SFRS->HF_OSC_TRIM.SSCDIV   = 15U;
    SYSCTRLA_SFRS->HF_OSC_TRIM.SSCENA   = 1U;

    systemClock = SYSTEM_CLOCK_HIGH_FREQ;
    systemClock = systemClock >> (uint8_t)divider;
}
```

```c
void Clock_SystemMainClockInit(uint8_t divider)
{
    CRGA_SFRS->SYSCLKCTRL.HFRCENA = 1U;
    while(CRGA_SFRS->SYSCLKCTRL.HFRCSTS == 0U){}
    /* Enable trim access write enable */
    SYSCTRLA_SFRS->TRIM_ACCESS_KEY.TRIM_ACCESS_KEY = 0x0EU;
    SYSCTRLA_SFRS->HF_OSC_TRIM.TRIM_HF_RC = HWCFG_GetRCHFClockCalibValue();
    PMUAPRE5V_SFRS->CLK_CTRL.TRIM_LF_RC = HWCFG_GetRCLFClockCalibValue();
    CRGA_SFRS->LFCLKCTRL.CLKLFSEL = 1;
    /* Disable trim access write enable */
    SYSCTRLA_SFRS->TRIM_ACCESS_KEY.TRIM_ACCESS_KEY = 0U;
    CRGA_SFRS->SYSCLKCTRL.SYSCLKSEL = (uint8_t)CLOCK_RC_16MHz;
    CRGA_SFRS->SYSCLKCTRL.DIVSYSCLK = (uint8_t)divider;
    /*  frequency spread    */
    SYSCTRLA_SFRS->HF_OSC_TRIM.SSCDEEP  = 7U;/*  3  */
    SYSCTRLA_SFRS->HF_OSC_TRIM.SSCDIV   = 15U;
    SYSCTRLA_SFRS->HF_OSC_TRIM.SSCENA   = 1U;

    systemClock = SYSTEM_CLOCK_HIGH_FREQ;
    systemClock = systemClock / (uint8_t)(divider + 1U);
}
```

You, 22小时前 • Uncommitted changes

4 V1.13

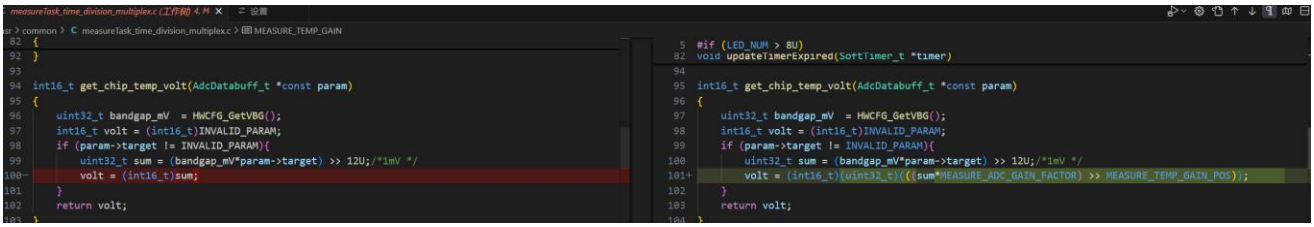
4.1 CHANGEVERSION

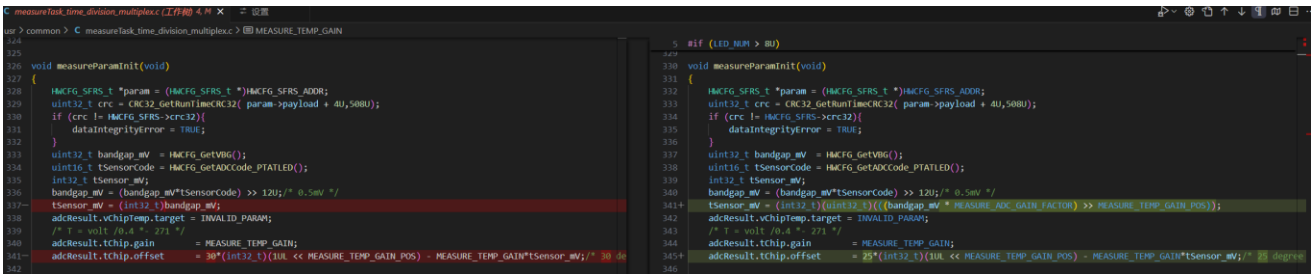
Form Version: libdev_iND83213A_CS01_App_B0.01.14_lin_2.1.0_colorlib_2.2.2_40leds

Update Version: libdev_iND83213A_CS01_App_B0.01.15_lin_2.1.0_colorlib_2.2.3_40leds

4.2 BUG FIX

4.2.1 Update Temperature Calculation Method





4.2.2 Add Handling Negative Voltage

```
}

int16_t get_volt(AdcDatabuff_t *const param)
{
    int16_t volt = (int16_t)INVALID_PARAM;
    if (param->target != INVALID_PARAM){
        int32_t sVolt = param->calibP.gain * (int32_t)param->target + param->calibP.offset;


        uint32_t uVolt = (uint32_t)sVolt;
        uVolt = uVolt >> MEASURE_GAIN_POS;
        volt = (int16_t)uVolt;
    }
    return volt;
}
```

```
#if (LED_NUM > 8U)

int16_t get_volt(AdcDatabuff_t *const param)
{
    int16_t volt = (int16_t)INVALID_PARAM;
    if (param->target != INVALID_PARAM){
        int32_t sVolt = param->calibP.gain * (int32_t)param->target + param->calibP.offset;
        if (sVolt <= 0){
            sVolt = 0;
        }
        uint32_t uVolt = (uint32_t)sVolt;
        uVolt = uVolt >> MEASURE_GAIN_POS;
        volt = (int16_t)uVolt;
    }
    return volt;
}
```

5 V1.12

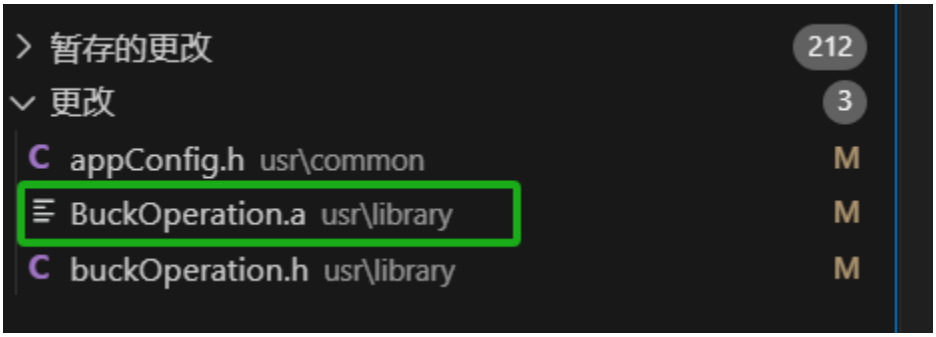
5.1 CHANGEVERSION

Form Version: libdev_iND83213A_CS01_App_B0.01.13_lin_2.1.0_colorlib_2.2.2_40leds

Update Version: libdev_iND83213A_CS01_App_B0.01.14_lin_2.1.0_colorlib_2.2.3_40leds

5.2 BUG FIX

5.2.1 Fix the issue of Buck Set Freq Api Inaccurate settings

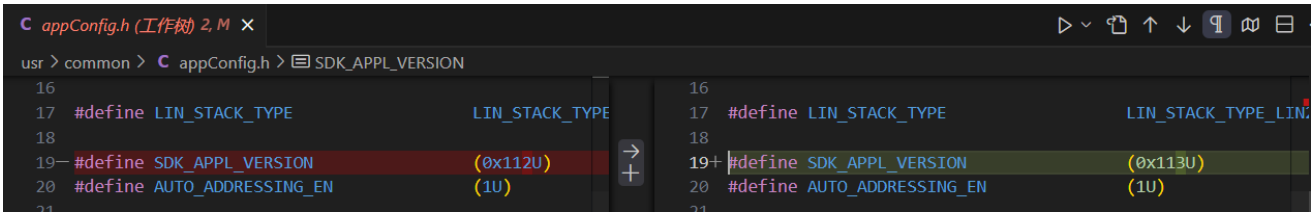


6 V1.11

6.1 Version Change

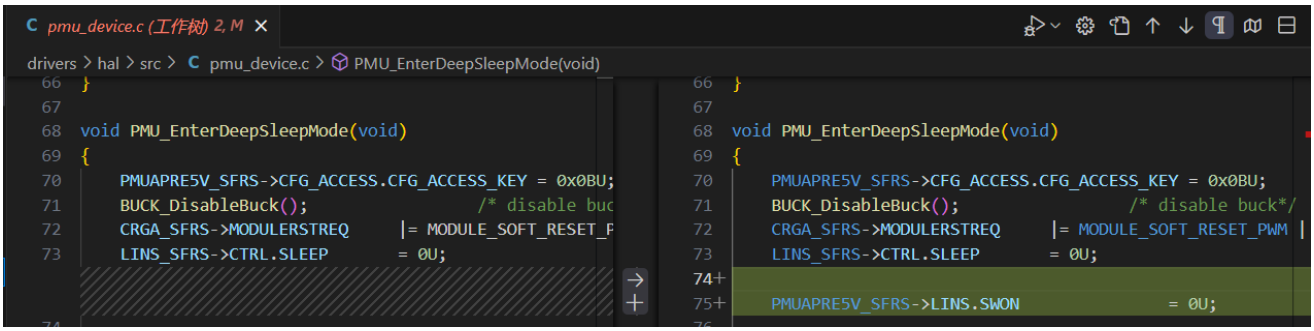
Form Version: libdev_iND83213A_CS01_App_B0.01.12_lin_2.1.0_colorlib_2.2.2_40leds

Update Version: libdev_iND83213A_CS01_App_B0.01.13_lin_2.1.0_colorlib_2.2.3_40leds



6.2 Feature Update

6.2.1 Update PMU_EnterDeepSleepMode(void) API



7 V1.10

7.1 VERSION CHANGE

Form Version: libdev_iND83213A_CS01_App_B0.01.11_lin_2.1.0_colorlib_2.2.2_40leds

Update Version: libdev_iND83213A_CS01_App_B0.01.12_lin_2.1.0_colorlib_2.2.3_40leds

```
appConfig.h (工作树) 4, M
usr > common > appConfig.h > SDK_APPL_VERSION
12  #define LIN_STACK_TYPE_LIN2_2A          (0U)
13  #define LIN_STACK_TYPE_SAEJ2602         (1U)
14  #define LIN_STACK_TYPE_PRIVATE          (2U)
15  #define LIN_STACK_TYPE_ISO17987         LIN_STACK_TYPE_LIN2_2A
16
17  #define LIN_STACK_TYPE                  LIN_STACK_TYPE_LIN2_2A
18
19- #define SDK_APPL_VERSION                (0x111U)
20  #define AUTO_ADDRESSING_EN              (1U)
21
```

```
12  #define LIN_STACK_TYPE_LIN2_2A          (0U)
13  #define LIN_STACK_TYPE_SAEJ2602         (1U)
14  #define LIN_STACK_TYPE_PRIVATE          (2U)
15  #define LIN_STACK_TYPE_ISO17987         LIN_STACK_TYPE_LIN2_2A
16
17  #define LIN_STACK_TYPE                  LIN_STACK_TYPE_LIN2_2A
18
19+ #define SDK_APPL_VERSION                (0x112U)
20  #define AUTO_ADDRESSING_EN              (1U)
21
```

7.2 NEW FEATURE

7.2.1 ADD HSL&RGB Color Mixing CMDS @ 40LEDS APP

```
applicationTask.c (工作树) 3, M
usr > common > applicationTask.c > APPL_HandleColorControlCommands(LIN_Device_Frame_t const *)
104 void APPL_HandleColorControlCommands(LIN_Device_Frame_t const *frame)
105 {
106     ColorCtrlFrame_t const *color = (ColorCtrlFrame_t const*)((void cc
107     uint8_t nodeAddress = color->nodeAddress & 0x3FU;
108     uint8_t ledNumMsb = color->nodeAddress >> 6U;
109     uint8_t ledNum = color->channel + (ledNumMsb << 4U);
110     if ( (nodeAddress == ls_read_nad()) || (nodeAddress == LIN_NAD_WIL
111         if (ledNum < LED_NUM){
112 //          LedNum_t ledIndex = color->channel;
113             LedNum_t ledIndex = (LedNum_t)ledNum;
114             switch(color->command){
115             case CMD_COLOR_CTRL_XYY:
116               (void)CLM_SetXYY(ledIndex,MES_GetLedTemperature(ledInde
117               break;
118             case CMD_COLOR_CTRL_XYY_EXT:
119               (void)CLM_SetXYY(ledIndex,MES_GetLedTemperature(ledInde
120               break;
121             case CMD_COLOR_CTRL_RGBL:
122               if (color->ColorRGBL.level > 0U){
123                   LED_State = 1U;
124               }else{
125                   LED_State = 0U;
126               }
127               (void)CLM_SetRGBL(ledIndex,MES_GetLedTemperature(ledInde
128               break;
129             case CMD_COLOR_CTRL_PWM:
130               (void)CLM_SetPWMs(ledIndex,(uint16_t)color->ColorPWM.rec
131               break;
132             case CMD_COLOR_CTRL_LUV:
133               (void)CLM_SetLUV(ledIndex,MES_GetLedTemperature(ledInde
134               break;
```

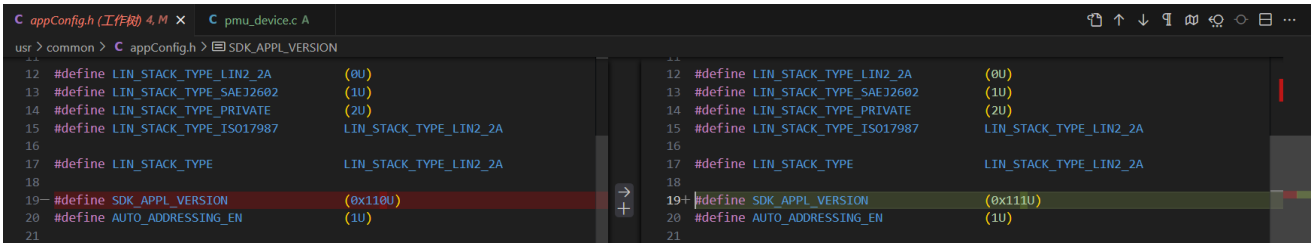
```
104 void APPL_HandleColorControlCommands(LIN_Device_Frame_t const *frame)
105 {
106     ColorCtrlFrame_t const *color = (ColorCtrlFrame_t const*)((void const
107     uint8_t nodeAddress = color->nodeAddress & 0x3FU;
108     uint8_t ledNumMsb = color->nodeAddress >> 6U;
109     uint8_t ledNum = color->channel + (ledNumMsb << 4U);
110     if ( (nodeAddress == ls_read_nad()) || (nodeAddress == LIN_NAD_WILDCA
111         if (ledNum < LED_NUM){
112 //          LedNum_t ledIndex = color->channel;
113             LedNum_t ledIndex = (LedNum_t)ledNum;
114             switch(color->command){
115             case CMD_COLOR_CTRL_XYY:
116               (void)CLM_SetXYY(ledIndex,MES_GetLedTemperature(ledIndex),
117               break;
118             case CMD_COLOR_CTRL_XYY_EXT:
119               (void)CLM_SetXYY(ledIndex,MES_GetLedTemperature(ledIndex),
120               break;
121+            case CMD_COLOR_CTRL_HSL:
122+              (void)CLM_SetHSL(ledIndex,MES_GetLedTemperature(ledIndex),
123+              break;
124             case CMD_COLOR_CTRL_RGBL:
125               if (color->ColorRGBL.level > 0U){
126                   LED_State = 1U;
127               }else{
128                   LED_State = 0U;
129               }
130               (void)CLM_SetRGBL(ledIndex,MES_GetLedTemperature(ledIndex),
131               break;
132+            case CMD_COLOR_CTRL_RGB:
133+              (void)CLM_SetRGB(ledIndex,MES_GetLedTemperature(ledIndex),
134+              break;
135             case CMD_COLOR_CTRL_PWM:
136               (void)CLM_SetPWMs(ledIndex,(uint16_t)color->ColorPWM.red*25
137               break;
138             case CMD_COLOR_CTRL_LUV:
139               (void)CLM_SetLUV(ledIndex,MES_GetLedTemperature(ledIndex),
140               break;
```

8 V1.09

8.1 Version Change

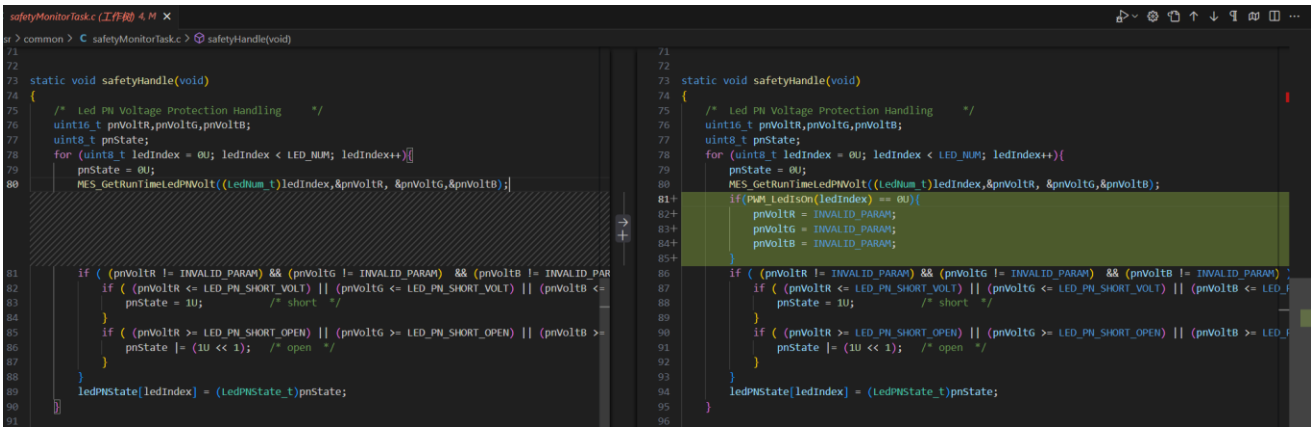
Form Version: libdev_iND83213A_CS01_App_B0.01.10_lin_2.0.9_colorlib_2.2.2_40leds

Update Version: libdev_iND83213A_CS01_App_B0.01.11_lin_2.1.0_colorlib_2.2.2_40leds



8.2 Bug Fix

8.2.1 Fix the issue of reporting led error when it is turned off



8.3 NEW FEATURE

8.3.1 Add an linstack api for Once Force 3D response without 3C require.

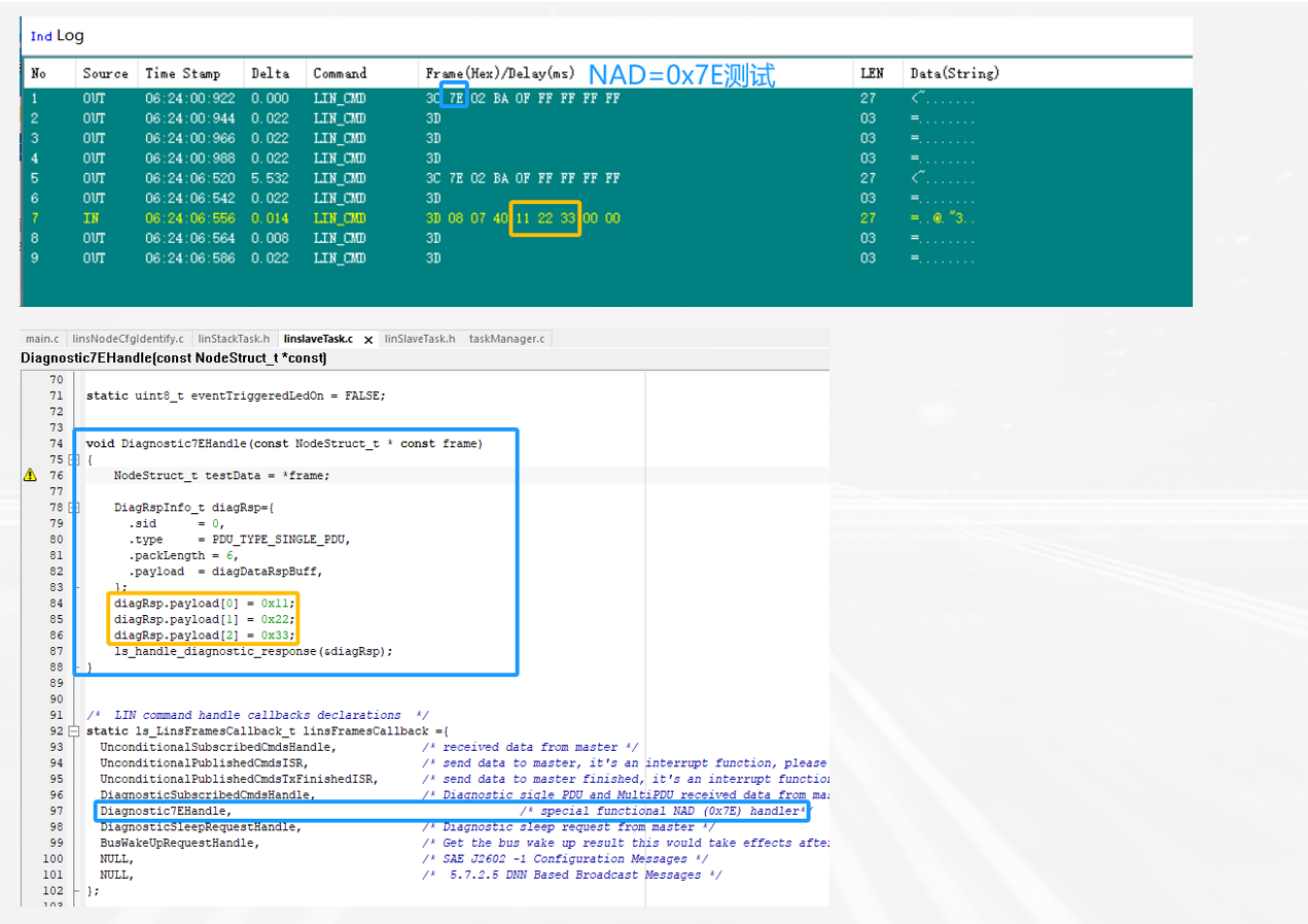
```
/*
 *  @return TxStatus_t.
 */
TxStatus_t ls_tx_status(void);

/*
```

```
 *  frame response send status.
 *  @return TxStatus_t.
 */
TxStatus_t ls_tx_status(void);

/*
 *  workaround: require 3D response when no 3C request.
 *  @param [in] nad: the NAD required.
 *  @return none
 */
void ls_set_init_req_nad(uint8_t nad);
/*
```

This Feature is used for boot mode and app mode switch. And before use this function, user must call the below api and prepare data into LIN frame buff.

```
void LINS_TaskHandler(void)
{
  DiagRspInfo_t diagRsp={
    .sid      = 0,
    .type     = PDU_TYPE_SINGLE_PDU,
    .packLength = 6,
    .payload    = diagDataRspBuff,
  };
    switch(linsTaskState){
    case TASK_STATE_ACTIVE:
      break;
    case TASK_STATE_INIT:
      ls_register_services(LIN_PROTOCOL_LIN2_2A,UnconditionalCmdsTable, (l_u8)(sizeof(UnconditionalCmdsTable)/sizeof(LIN_Device_Frame_t)), DIAG_DATA_BUFF_SIZE,&linsFramesCall
      (void)ls_set_tp_timeout(N_AS, N_CR);
      (void)l_sys_init();

      ls_set_init_req_nad(0x05);
      ls_handle_diagnostic_response(&diagRsp);

      linsTaskState = TASK_STATE_ACTIVE;
      break;
    default:
      break;
    }
}
```

8.3.2 Add special functional NAD (0x7E) handler.



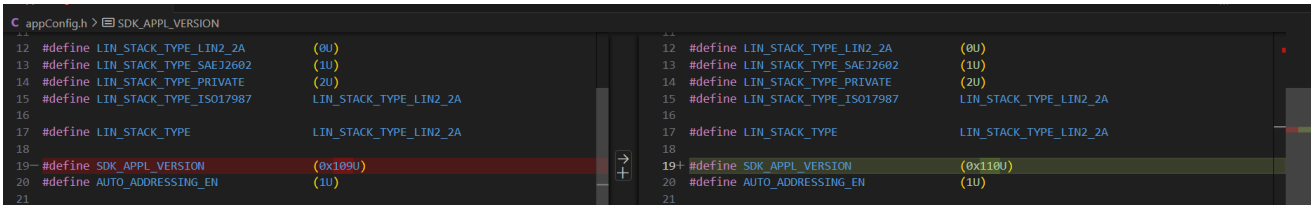
This Feature is used for special functional NAD (0x7E). And before use this function, user must call the below api and prepare data into LIN frame buff.

9 V1.08

9.1 Version Change

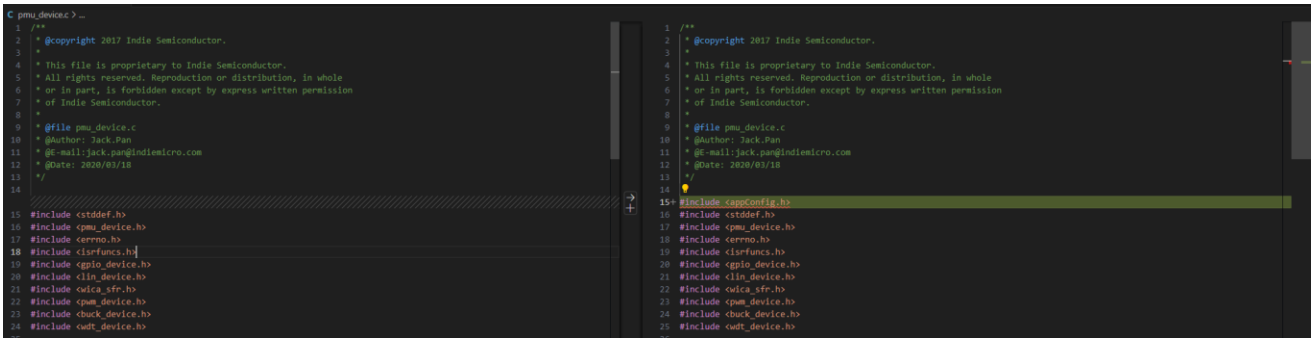
Form Version: libdev_iND83213A_CS01_App_B0.01.09_lin_2.0.9_colorlib_2.2.2_40leds

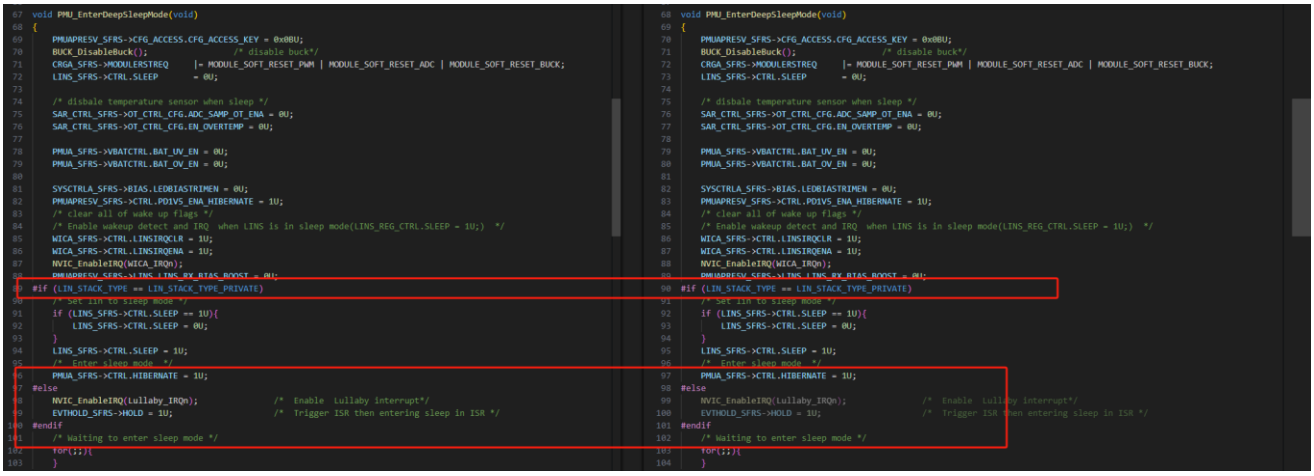
Update Version: libdev_iND83213A_CS01_App_B0.01.10_lin_2.0.9_colorlib_2.2.2_40leds



9.2 BUG Fix

9.2.1 Resolve the issue of abnormal sleep mode branches



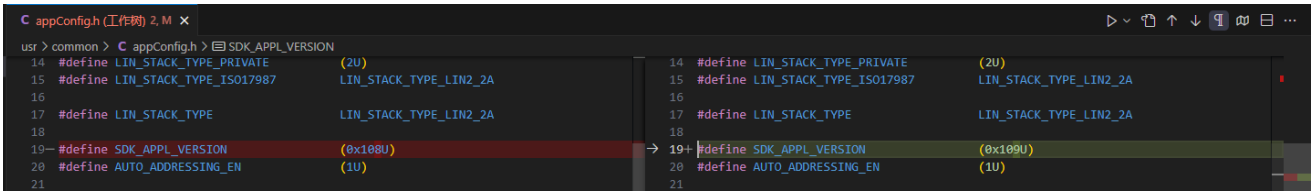


10V1.07

10.1CHANGEVERSION

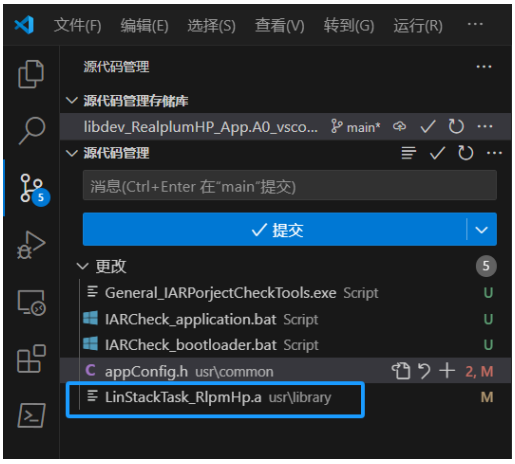
Form Version: libdev_iND83213A_CS01_App_B0.01.08_lin_2.0.8_colorlib_2.2.2_40leds

Update Version: libdev_iND83213A_CS01_App_B0.01.09_lin_2.0.9_colorlib_2.2.2_40leds



10.2LIN STACK UPDATE

10.2.1 Update error report logic @J2602 Mode



10.3BUG FIXED

10.3.1 Update LIN device Params for improve LIN_AA performance

lin_device.c (工作树) 2, M

drivers > hal > src > lin_device.c > LINS_Init(void)

```
121 #endif
123     LINS_SFRS->BUSTIME.WUPREPEAT        = E_LIN_TIME_WAKEUP_REPEAT_MS_200;
124     LINS_SFRS->CTRL.RST_INT_ERR         = 0x03U;    /* reset error,reset interrupt
125
126     PMUAPRE5V_SFRS->LINS.LINS_TX_BIAS_BOOST = (uint8_t)HWCFG_GetLINS_TxBiasBoost();
127     PMUAPRE5V_SFRS->LINM.LINM_TX_BIAS_BOOST = (uint8_t)HWCFG_GetLINM_TxBiasBoost();
128     PMUAPRE5V_SFRS->LINS.LINS_RX_BIAS_BOOST = 0x03U;
129     PMUAPRE5V_SFRS->LINM.LINM_RX_BIAS_BOOST = 0x03U;
130
131 #if AUTO_ADDRESSING_EN == 0U
132     PMUAPRE5V_SFRS->LINS.SWON               = 0U; /* 1: LIN slave switch On between
133
134     PMUAPRE5V_SFRS->LINS.LIN_TX_S_SL        = (uint8_t)HWCFG_GetLINS_SlewRate();
135     PMUAPRE5V_SFRS->LINS.LIN_TX_S_DT        = (uint8_t)HWCFG_GetLINS_TxDuty();
136 #else
137     PMUAPRE5V_SFRS->LINM.LIN_TX_M_SL        = 0x0F;
138-    PMUAPRE5V_SFRS->LINS.LIN_TX_S_DT        = 0x03U;
139     PMUAPRE5V_SFRS->LINS.LIN_TX_S_SL        = 0x0F;
140-    PMUAPRE5V_SFRS->LINM.LIN_TX_M_DT        = 0x03U;
141
142     PMUAPRE5V_SFRS->LINS.SWON               = 1U; /* 1: LIN slave switch On between
143     PMUAPRE5V_SFRS->LINM.LINM_PUOFF_TIMEOUT = 0U;
144 #endif
```

```
105 void LINS_Init(void)
123     LINS_SFRS->BUSTIME.WUPREPEAT        = E_LIN_TIME_WAKEUP_REPEAT_MS_200;
124     LINS_SFRS->CTRL.RST_INT_ERR         = 0x03U;    /* reset error,reset interrupt
125
126     PMUAPRE5V_SFRS->LINS.LINS_TX_BIAS_BOOST = (uint8_t)HWCFG_GetLINS_TxBiasBoost();
127     PMUAPRE5V_SFRS->LINM.LINM_TX_BIAS_BOOST = (uint8_t)HWCFG_GetLINM_TxBiasBoost();
128     PMUAPRE5V_SFRS->LINS.LINS_RX_BIAS_BOOST = 0x03U;
129     PMUAPRE5V_SFRS->LINM.LINM_RX_BIAS_BOOST = 0x03U;
130
131 #if AUTO_ADDRESSING_EN == 0U
132     PMUAPRE5V_SFRS->LINS.SWON               = 0U; /* 1: LIN slave switch On between
133
134     PMUAPRE5V_SFRS->LINS.LIN_TX_S_SL        = (uint8_t)HWCFG_GetLINS_SlewRate();
135     PMUAPRE5V_SFRS->LINS.LIN_TX_S_DT        = (uint8_t)HWCFG_GetLINS_TxDuty();
136 #else
137     PMUAPRE5V_SFRS->LINM.LIN_TX_M_SL        = 0x0F;
138+    PMUAPRE5V_SFRS->LINS.LIN_TX_S_DT        = 0x02U;
139     PMUAPRE5V_SFRS->LINS.LIN_TX_S_SL        = 0x0F;
140+    PMUAPRE5V_SFRS->LINM.LIN_TX_M_DT        = 0x02U;
141
142     PMUAPRE5V_SFRS->LINS.SWON               = 1U; /* 1: LIN slave switch On between
143     PMUAPRE5V_SFRS->LINM.LINM_PUOFF_TIMEOUT = 0U;
144 #endif
```

10.3.2 Fix pwm device driver API error

pwm_device.c (工作树) 2, M

drivers > hal > src > pwm_device.c > PWM_Handler(void)

```
33 static uint16_t currPWM[24U];
34 static PwmIsrCallback_t pwmReloadCallback = NULL;
35
36 void PWM_Handler(void)
37 {
38-    if (pwmReloadCallback == NULL){
39         pwmReloadCallback();
40     }
41 }
```

```
33 static uint16_t currPWM[24U];
34 static PwmIsrCallback_t pwmReloadCallback = NULL;
35
36 void PWM_Handler(void)
37 {
38+    if (pwmReloadCallback != NULL){
39         pwmReloadCallback();
40     }
41 }
```

pwm_device.c (工作树) 2, M

drivers > hal > src > pwm_device.c > PWM_Handler(void)

```
130 void PWM_RegisterPWMReloadIRQ(PwmIsrCallback_t callback)
131 {
132     pwmReloadCallback = callback;
133     PWM_SFRS->INTPWM.CLEAR.PERIOD = 1U;
134-    PWM_SFRS->INTPWM.IRQ.PERIOD = 1U;
135     PWM_SFRS->UPDATE = 1U;
136     NVIC_EnableIRQ(PWM_IRQn);
137 }
138
139 /**
140  * @brief The instance of PMW device
141  */
142 void PWM_UnRegisterPWMReloadIRQ(void)
143 {
144     pwmReloadCallback = NULL;
145     PWM_SFRS->INTPWM.CLEAR.PERIOD = 1U;
146-    PWM_SFRS->INTPWM.IRQ.PERIOD    = 0U;
147     PWM_SFRS->UPDATE = 1U;
148     NVIC_DisableIRQ(PWM_IRQn);
149 }
```

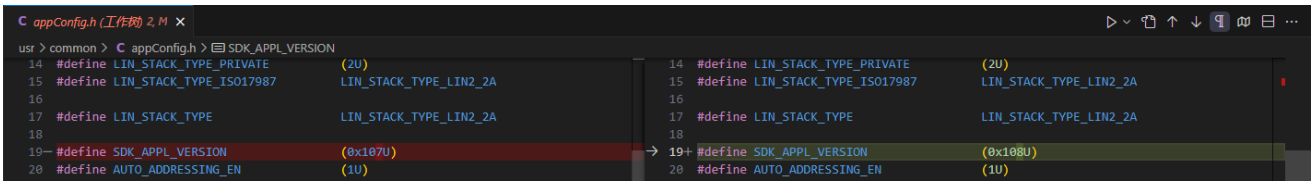
```
130 void PWM_RegisterPWMReloadIRQ(PwmIsrCallback_t callback)
131 {
132     pwmReloadCallback = callback;
133     PWM_SFRS->INTPWM.CLEAR.PERIOD = 1U;
134+    PWM_SFRS->INTPWM.ENABLE.PERIOD = 1U;
135     PWM_SFRS->UPDATE = 1U;
136     NVIC_EnableIRQ(PWM_IRQn);
137 }
138
139 /**
140  * @brief The instance of PMW device
141  */
142 void PWM_UnRegisterPWMReloadIRQ(void)
143 {
144     pwmReloadCallback = NULL;
145     PWM_SFRS->INTPWM.CLEAR.PERIOD = 1U;
146+    PWM_SFRS->INTPWM.ENABLE.PERIOD    = 0U;
147     PWM_SFRS->UPDATE = 1U;
148     NVIC_DisableIRQ(PWM_IRQn);
149 }
```

11 V1.06

11.1 CHANGE VERSION

Form Version: libdev_iND83213A_CS01_App_B0.01.07_lin_2.0.8_colorlib_2.2.2_40leds

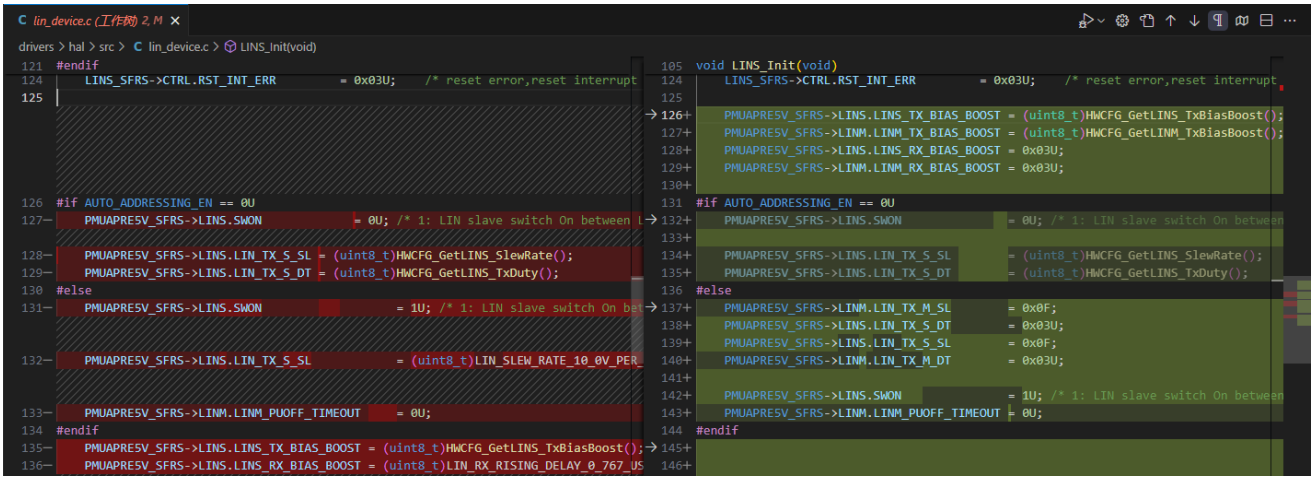
Update Version: libdev_iND83213A_CS01_App_B0.01.08_lin_2.0.8_colorlib_2.2.2_40leds



11.2 BUG FIXED

11.2.1 Update LIN device Params for improve LIN_AA performance

New configuration can improve LIN Duty after multiple nodes.



11.2.2 Fix sleep issue @Private_linsavetask mode

pmu_device.c (工作树) 2, M

drivers > hal > src > pmu_device.c > PMU_EnterDeepSleepMode(void)

```c
{

    /* disbale temperature sensor when sleep */
    SAR_CTRL_SFRS->OT_CTRL_CFG.ADC_SAMP_OT_ENA = 0U;
    SAR_CTRL_SFRS->OT_CTRL_CFG.EN_OVERTEMP = 0U;

    PMUA_SFRS->VBATCTRL.BAT_UV_EN = 0U;
    PMUA_SFRS->VBATCTRL.BAT_OV_EN = 0U;

    SYSCTRLA_SFRS->BIAS.LEDBIASTRIMEN = 0U;
    PMUAPRE5V_SFRS->CTRL.PD1V5_ENA_HIBERNATE = 0U;
    /* clear all of wake up flags */
    /* Enable wakeup detect and IRQ  when LINS is in sleep mode(LINS_REG_CTRL.S
    WICA_SFRS->CTRL.LINSIRQCLR = 1U;
    WICA_SFRS->CTRL.LINSIRQENA = 1U;
    NVIC_EnableIRQ(WICA_IRQn);
    PMUAPRE5V_SFRS->LINS.LINS_RX_BIAS_BOOST = 0U;

    NVIC_EnableIRQ(Lullaby_IRQn);               /*  Enable  Lullaby interrupt*/
    EVTHOLD_SFRS->HOLD = 1U;                    /*  Trigger ISR then entering s

    /* Waiting to enter sleep mode */
    for(;;){
    }
}
```

```c
void PMU_EnterDeepSleepMode(void)

    /* disbale temperature sensor when sleep */
    SAR_CTRL_SFRS->OT_CTRL_CFG.ADC_SAMP_OT_ENA = 0U;
    SAR_CTRL_SFRS->OT_CTRL_CFG.EN_OVERTEMP = 0U;

    PMUA_SFRS->VBATCTRL.BAT_UV_EN = 0U;
    PMUA_SFRS->VBATCTRL.BAT_OV_EN = 0U;

    SYSCTRLA_SFRS->BIAS.LEDBIASTRIMEN = 0U;
    PMUAPRE5V_SFRS->CTRL.PD1V5_ENA_HIBERNATE = 1U;
    /* clear all of wake up flags */
    /* Enable wakeup detect and IRQ  when LINS is in sleep mode(LINS_REG_CTRL.S
    WICA_SFRS->CTRL.LINSIRQCLR = 1U;
    WICA_SFRS->CTRL.LINSIRQENA = 1U;
    NVIC_EnableIRQ(WICA_IRQn);
    PMUAPRE5V_SFRS->LINS.LINS_RX_BIAS_BOOST = 0U;
#if (LIN_STACK_TYPE == LIN_STACK_TYPE_PRIVATE)
    /* Set lin to sleep mode */
    if (LINS_SFRS->CTRL.SLEEP == 1U){
        LINS_SFRS->CTRL.SLEEP = 0U;
    }
    LINS_SFRS->CTRL.SLEEP = 1U;
    /*  Enter sleep mode  */
    PMUA_SFRS->CTRL.HIBERNATE = 1U;
#else
    NVIC_EnableIRQ(Lullaby_IRQn);               /*  Enable  Lullaby interrupt*/
    EVTHOLD_SFRS->HOLD = 1U;                    /*  Trigger ISR then entering s
#endif
    /* Waiting to enter sleep mode */
    for(;;){
    }
}
```

12 V1.05

12.1 CHANGE VERSION

Form Version: libdev_iND83213A_CS01_App_B0.01.06_lin_2.0.8_colorlib_2.2.2_40leds

Update Version: libdev_iND83213A_CS01_App_B0.01.07_lin_2.0.8_colorlib_2.2.2_40leds

```
appConfig.h (工作树) 2, M
usr > common > appConfig.h > SDK_APPL_VERSION
17  #define LIN_STACK_TYPE                  LIN_STACK_TYPE_LIN2_2A
18
19- #define SDK_APPL_VERSION                (0x106U)
20  #define AUTO_ADDRESSING_EN              (1U)
```

```
17  #define LIN_STACK_TYPE                  LIN_STACK_TYPE_LIN2_2A
18
19+ #define SDK_APPL_VERSION                (0x107U)
20  #define AUTO_ADDRESSING_EN              (1U)
```

12.2 BUG FIXED

12.2.1 Porting linslaveTask_Private.c

```
lins_sfr.h (工作树) 1, M
drivers > hdf > sfrs > lins_sfr.h > LINS_REG_DATA(index)
196  */
197  #define LINS_SFRS ((__IO LINS_SFRS_t *)0x50010300)
```

```
196  */
197+ #define LINS_REG_DATA(index)          (*(__IO          uint8_t *)(0x50010300 + index)*
198+ #define LINS_REG_STATUS               (*(__IO          uint8_t *)(0x50010324))
199+ #define LINS_REG_DL                   (*(__IO          uint8_t *)(0x5001032C))
200+ #define LINS_REG_ID                   (*(__IO          uint8_t *)(0x50010338))
201  #define LINS_SFRS ((__IO LINS_SFRS_t *)0x50010300)
```

```
lin_device.c (工作树) 2, M
drivers > hal > src > lin_device.c > ...
350  static __INLINE void dataReceivedACK(uint8_t dataLength, ChecksumType_t checksumType)
351  {
352      LINS_REG_DL = dataLength + ((uint8_t)checksumType << LINS_DL_ENHCHK_POS) + (0U << LIN
353-     LINS_REG_CTRL.DATAACK_TPYE = DATAACK_TPYE_RECEIVE;/* it's a data receive ack operatio
354  }
355
356  static __INLINE void dataResponseACK(uint8_t* payload, uint8_t dataLength,ChecksumType_t
357  {
358      LINS_REG_DL = dataLength + ((uint8_t)checksumType << LINS_DL_ENHCHK_POS) + (0U << LIN
359      for (uint8_t i = 0U; i < dataLength; i++){
360          LINS_REG_DATA(i) = payload[i];
361      }
362-     LINS_REG_CTRL.DATAACK_TPYE = DATAACK_TPYE_TRANSMIT;/* it's a data transmit ack operat
363  }
```

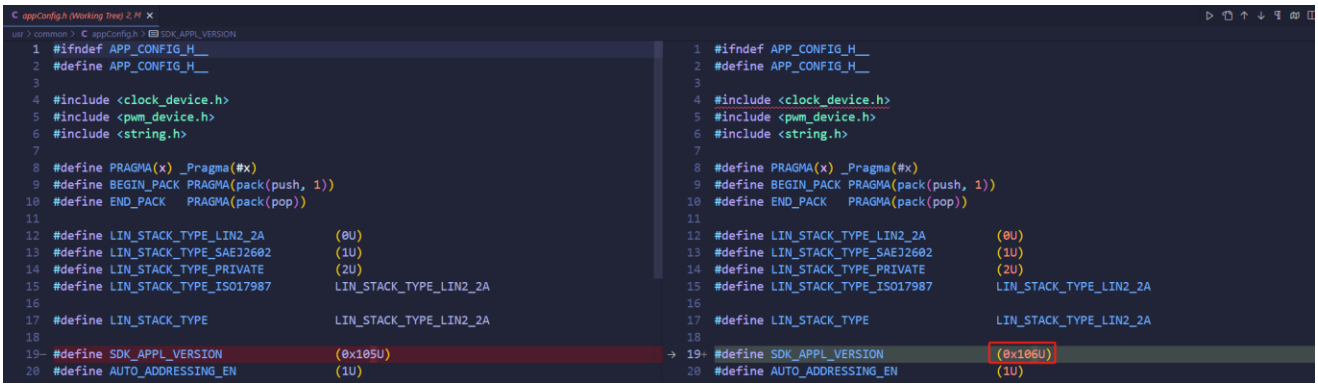
```
351  static __INLINE void dataReceivedACK(uint8_t dataLength, ChecksumType_t checksumType)
352  {
353      LINS_REG_DL = dataLength + ((uint8_t)checksumType << LINS_DL_ENHCHK_POS) + (0U << LIN
354+     LINS_SFRS->CTRL.DATAACK_TPYE = DATAACK_TPYE_RECEIVE;/* it's a data receive ack operat
355  }
356
357  static __INLINE void dataResponseACK(uint8_t* payload, uint8_t dataLength,ChecksumType_t
358  {
359      LINS_REG_DL = dataLength + ((uint8_t)checksumType << LINS_DL_ENHCHK_POS) + (0U << LIN
360      for (uint8_t i = 0U; i < dataLength; i++){
361          LINS_REG_DATA(i) = payload[i];
362      }
363+     LINS_SFRS->CTRL.DATAACK_TPYE = DATAACK_TPYE_TRANSMIT;/* it's a data transmit ack oper
364  }
```

13V1.04

13.1CHANGEVERSION

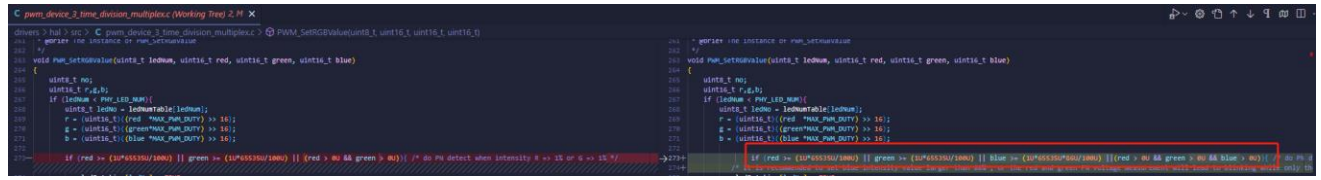
Form Version: libdev_iND83213A_CS01_App_B0.01.05_lin_2.0.8_colorlib_2.2.2_40leds

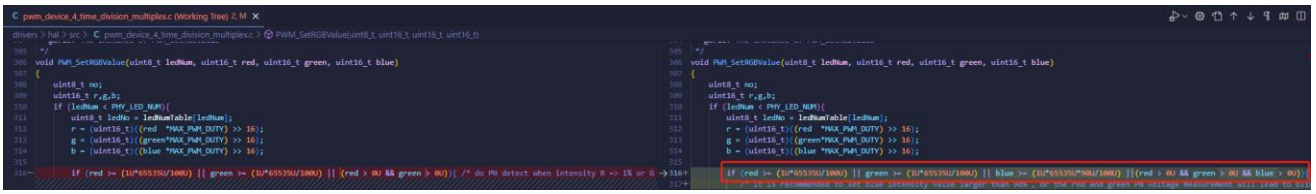
Update Version: libdev_iND83213A_CS01_App_B0.01.06_lin_2.0.8_colorlib_2.2.2_40leds



13.2BUG FIXED

13.2.1 Set blue intensity threshold value before PN detection



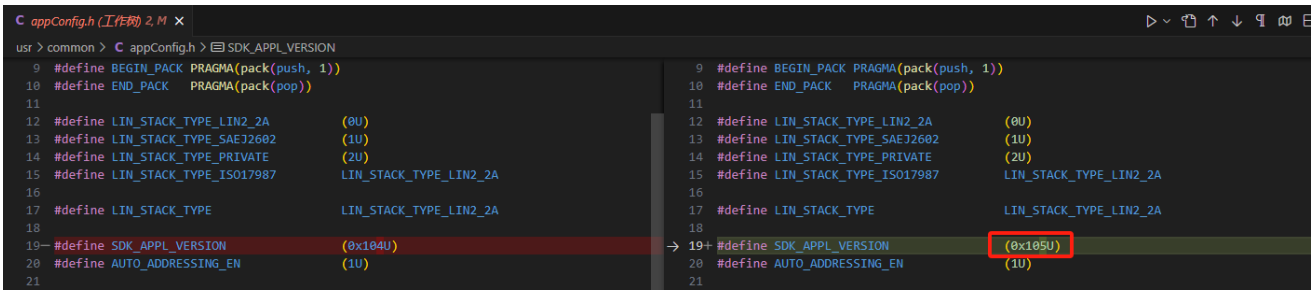


14V1.03

14.1CHANGEVERSION

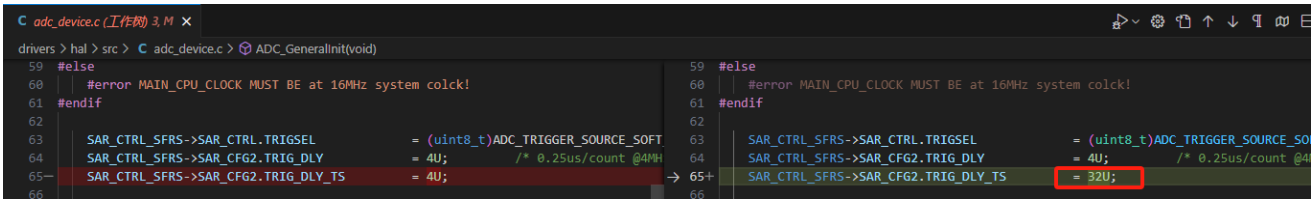
Form Version: libdev_iND83213A_CS01_App_B0.01.04_lin_2.0.8_colorlib_2.2.2_40leds

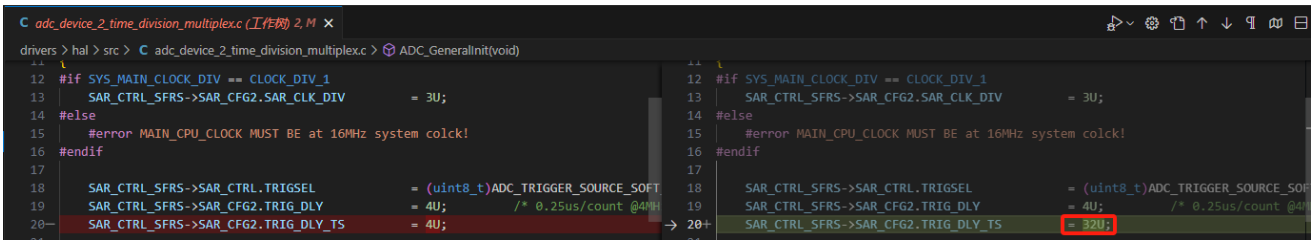
Update Version: libdev_iND83213A_CS01_App_B0.01.05_lin_2.0.8_colorlib_2.2.2_40leds



14.2DEFAULT STATUS MODIFY

14.2.1 Default Params Modify For SNPD



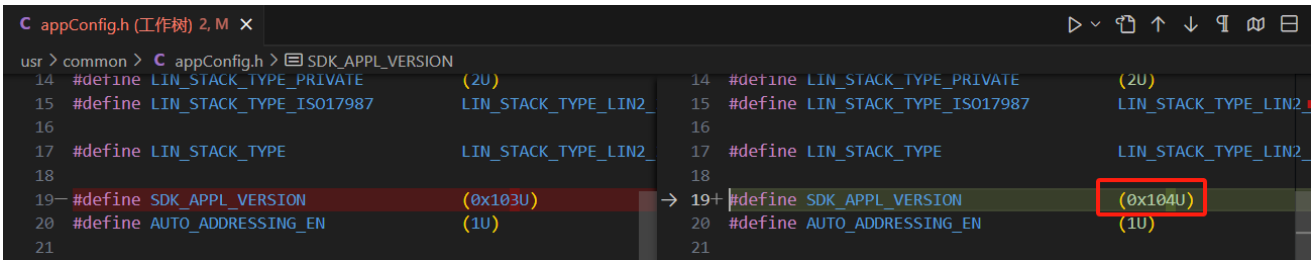


15 V1.02

15.1 CHANGE VERSION

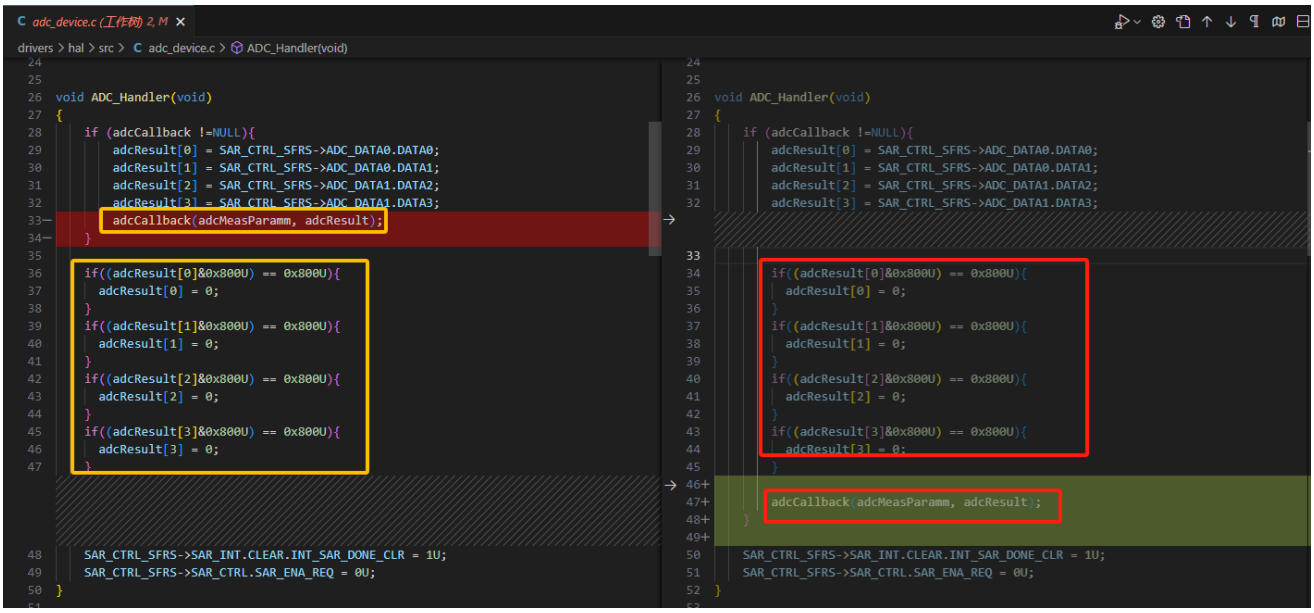
Form Version: libdev_iND83213A_CS01_App_B0.01.03_lin_2.0.8_colorlib_2.2.2_40leds

Update Version: libdev_iND83213A_CS01_App_B0.01.04_lin_2.0.8_colorlib_2.2.2_40leds



15.2 BUG FIXED

15.2.1 ADC negative processing before calling adcCallback

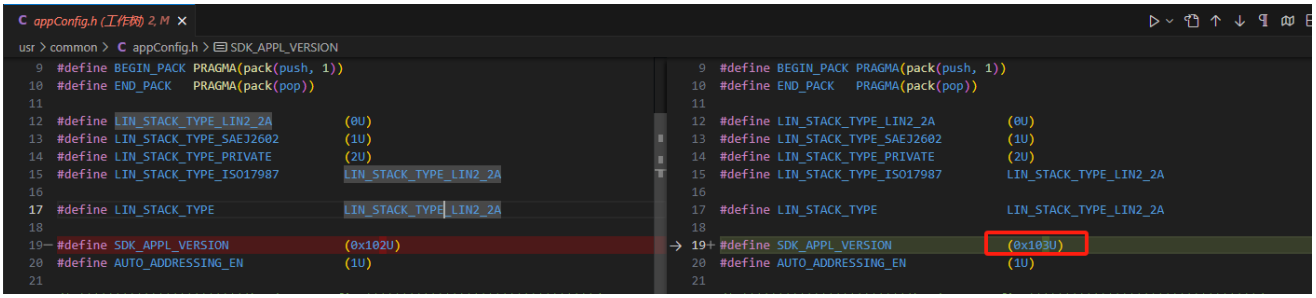


16 V1.01

16.1 CHANGE VERSION

Form Version: libdev_iND83213A_CS01_App_B0.01.02_lin_2.0.8_colorlib_2.2.2_40leds

Update Version: libdev_iND83213A_CS01_App_B0.01.03_lin_2.0.8_colorlib_2.2.2_40leds



16.2 BUG FIXED

16.2.1 None

16.3New Feature

16.3.1 Add ADC Code Filter

Force negative adc code to be 0.

Adc_decices.c

```
adc_device.c (工作树) 2, M
drivers > hal > src > adc_device.c > ADC_Handler(void)

void ADC_Handler(void)
{
    if (adcCallback !=NULL){
        adcResult[0] = SAR_CTRL_SFRS->ADC_DATA0.DATA0;
        adcResult[1] = SAR_CTRL_SFRS->ADC_DATA0.DATA1;
        adcResult[2] = SAR_CTRL_SFRS->ADC_DATA1.DATA2;
        adcResult[3] = SAR_CTRL_SFRS->ADC_DATA1.DATA3;
        adcCallback(adcMeasParamm, adcResult);
    }

    SAR_CTRL_SFRS->SAR_INT.CLEAR.INT_SAR_DONE_CLR = 1U;
    SAR_CTRL_SFRS->SAR_CTRL.SAR_ENA_REQ = 0U;
}
```

```
void ADC_Handler(void)
{
    if (adcCallback !=NULL){
        adcResult[0] = SAR_CTRL_SFRS->ADC_DATA0.DATA0;
        adcResult[1] = SAR_CTRL_SFRS->ADC_DATA0.DATA1;
        adcResult[2] = SAR_CTRL_SFRS->ADC_DATA1.DATA2;
        adcResult[3] = SAR_CTRL_SFRS->ADC_DATA1.DATA3;
        adcCallback(adcMeasParamm, adcResult);
    }
+   if((adcResult[0]&0x800U) == 0x800U){
+     adcResult[0] = 0;
+   }
+   if((adcResult[1]&0x800U) == 0x800U){
+     adcResult[1] = 0;
+   }
+   if((adcResult[2]&0x800U) == 0x800U){
+     adcResult[2] = 0;
+   }
+   if((adcResult[3]&0x800U) == 0x800U){
+     adcResult[3] = 0;
+   }
    SAR_CTRL_SFRS->SAR_INT.CLEAR.INT_SAR_DONE_CLR = 1U;
    SAR_CTRL_SFRS->SAR_CTRL.SAR_ENA_REQ = 0U;
}
```

pwm_device_2_time_division_multiplex.c

pwm_device_3_time_division_multiplex.c

pwm_device_4_time_division_multiplex.c

pwm_device_5_time_division_multiplex.c

```
pwm_device_2_time_division_multiplex.c (工作树) 2, M
drivers > hal > src > pwm_device_2_time_division_multiplex.c > ADC_Handler(void)

    PWM_SFRS->UPDATE = 1U;

    if(ledConvertType == ADC_MEAS_TYPE_LEDPN_VBAT_VBUCK_PTATLED){
        adcCode = SAR_CTRL_SFRS->ADC_DATA0.DATA0;
        vLedPNCode[lastLedNo][lastLedChannel]   = adcCode;
    }
    adcCode = SAR_CTRL_SFRS->ADC_DATA0.DATA1; vBattCode = adcCode;
    adcCode = SAR_CTRL_SFRS->ADC_DATA1.DATA2; vBuckCode = adcCode;
    adcCode = SAR_CTRL_SFRS->ADC_DATA1.DATA3; vTempCode = adcCode;
```

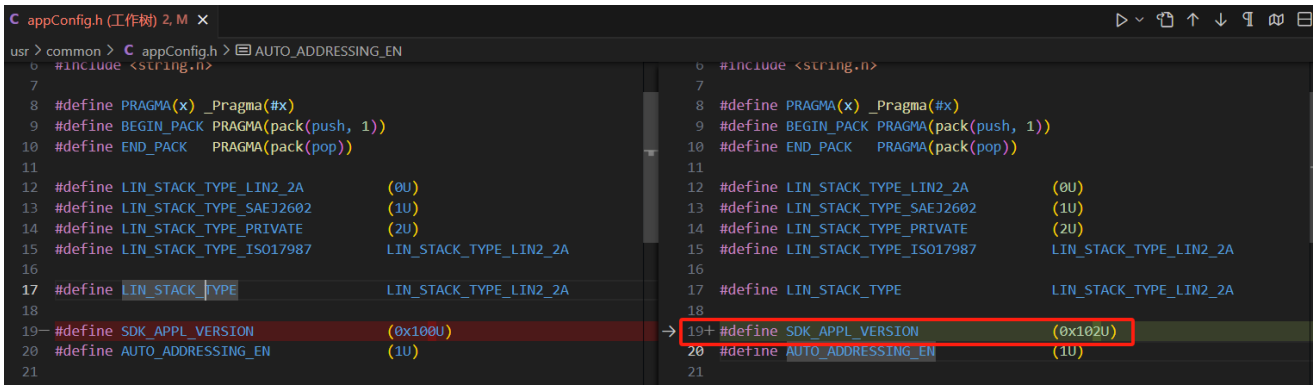
```
    PWM_SFRS->UPDATE = 1U;

    if(ledConvertType == ADC_MEAS_TYPE_LEDPN_VBAT_VBUCK_PTATLED){
        adcCode = SAR_CTRL_SFRS->ADC_DATA0.DATA0;
+       if((adcCode&0x800U) == 0x800U){
+         adcCode = 0;
+       }
        vLedPNCode[lastLedNo][lastLedChannel]   = adcCode;
    }
    adcCode = SAR_CTRL_SFRS->ADC_DATA0.DATA1; vBattCode = adcCode;
    adcCode = SAR_CTRL_SFRS->ADC_DATA1.DATA2; vBuckCode = adcCode;
    adcCode = SAR_CTRL_SFRS->ADC_DATA1.DATA3; vTempCode = adcCode;

+   if((vBattCode&0x800U) == 0x800U){
+     vBattCode = 0;
+   }
+   if((vBuckCode&0x800U) == 0x800U){
+     vBuckCode = 0;
+   }
+   if((vTempCode&0x800U) == 0x800U){
+     vTempCode = 0;
+   }
```

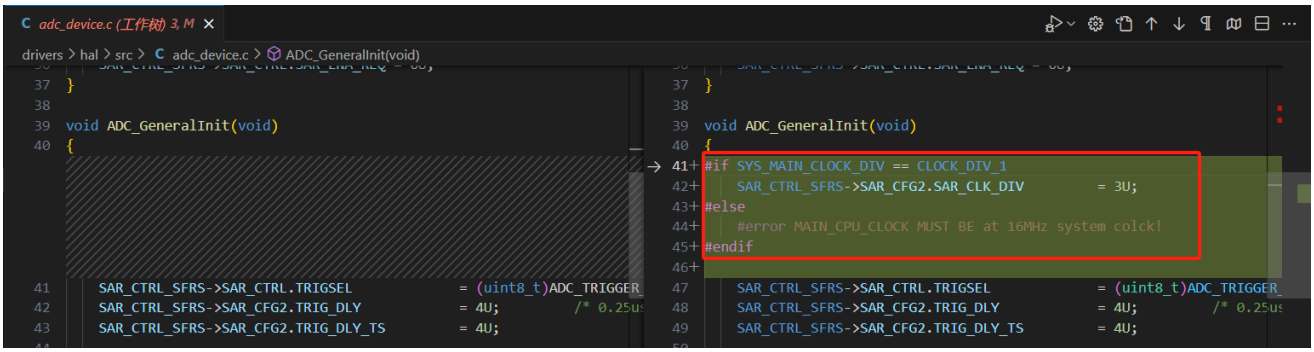
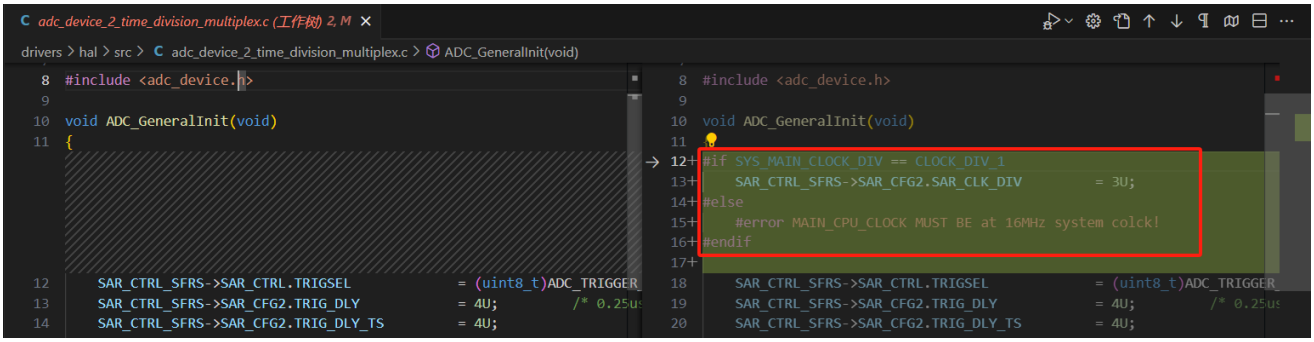
17V1.00

17.1CHANGEVERSION

Form Version: libdev_iND83213A_CS01_App_B0.01.01_lin_2.0.8_colorlib_2.2.2_40leds

Update Version: libdev_iND83213A_CS01_App_B0.01.02_lin_2.0.8_colorlib_2.2.2_40leds



17.2BUG FIXED

17.2.1 Make ADC clock source to be 4MHz